

mytools

A subset of bedtools, rebuilt in Python against the real thing as oracle
Luis R Arriola · natoarriola/ai_agent_workshop · 11 September 2026

What this is

mytools reimplements four things bedtools does - sort, merge, intersect, subtract - over BED interval files. Python 3, standard library only, one executable file.

The method matters more than the code. Real bedtools is installed on the VM and is treated as the **oracle**: every subcommand is diffed against it byte for byte on the same input, and where we disagree, we are wrong. No expected-output file was ever written by hand, so no misunderstanding could be enshrined in a test.

Where it stands

	Status	Detail
Golden tests	17 / 20	vs. real bedtools; the 3 remaining are subtract
Unit tests	102 passing	stdlib unittest, 4 ms, no bedtools needed
Lint	clean	ruff 0.16.7 pinned, plus shellcheck on the test runner
CI	never run	Actions needs enabling on the fork - see Outstanding
Merged to main	nothing yet	6 pull requests open in a stack

Subcommands

Command	Flags	State
sort	-i	done - 4/4 golden
merge	-i, -d	done - 6/6 golden
intersect	-a, -b, -u, -v, -wa	done - 7/7 golden
subtract	-a, -b	in progress - 0/3 golden
closest	-	out of scope for v1

Scope was cut deliberately. `closest` was dropped from v1 when the spec was adopted, on the principle that four finished subcommands beat seven half-built ones.

What the oracle actually does

This is the part worth keeping. BED is 0-based and half-open, so two intervals overlap only when `a.start < b.end` and `b.start < a.end`, strictly. Every off-by-one lives on that line. But zero-length intervals - legal in BED, and present in the fixtures on purpose - break the rule in a way nobody predicts:

bedtools widens every zero-length interval $[x, x)$ to $[x-1, x+1)$ before testing overlap. Confirmed on 12 probes before a line was written. What each subcommand then does with that widening is *different in all three cases*:

Subcommand	What the widening affects	Example the oracle gives
intersect	the decision only; reported coordinates mix -a's original with -b's widened	<code>chr1 100 200</code> $\cap$ <code>chr1 150 150</code> $\rightarrow$ <code>chr1 149 151</code>
merge	the decision and the output - except a cluster of one feature, which prints unchanged	<code>chr1 0 0</code> beside <code>chr1 0 10</code> $\rightarrow$ <code>chr1 -1 10</code> (negative!)
sort	nothing - ordering never consults it	<code>chr1 500 500</code> sorts before <code>chr1 500 600</code>

Four more findings that cost real debugging

Finding	Why it matters
intersect reports hits in -b file order , not coordinate order	Sorting the hits looks tidier and fails two golden cases.
bedtools crashes on a zero-length feature at position 0 in -b	It widens to -1 and its bin tree rejects it. A golden case pins that exit code, so mytools refuses the same input rather than succeeding where the oracle dies.
merge checks sortedness loosely	Start order within a chromosome only. <code>chr2</code> before <code>chr1</code> exits 0; only revisiting a chromosome is an error.
Chromosomes sort byte-wise , not naturally	<code>chr10</code> before <code>chr2</code> , and <code>chr17</code> before <code>chr7</code> .

How the work was done

#	Issue / PR	Outcome
1 / 5	Warm-up: mytools --version	usage to stderr, exit 2 with no arguments
2 / 4	Adopt or write a spec	SPEC.md adopted from the fallback, then corrected twice from evidence
3 / 6	Tests, linter, CI	20 golden cases, 30 unit cases, ruff, shellcheck, ci.yml
7 / 8	sort	stable, byte-wise chromosome order, verbatim column preservation
9 / 10	merge	streaming clusters; matched on 400 random inputs
11 / 12	intersect	bisect index over -b; matched on 300 random inputs

Two spec corrections came out of implementation, not planning. SPEC §3 claimed ragged column counts were tolerated - bedtools refuses them outright. And merge streams, so a chromosome revisit discovered mid-file leaves already-completed clusters on stdout where bedtools writes nothing; that one is recorded in §8 as an accepted deviation, with its reason, rather than quietly fixed.

Outstanding

What	Needs
Enable Actions on the fork	GitHub disables workflows on forks until the owner clicks through the banner on the Actions tab. CI has therefore never run: all three guardrails are verified locally only.
Merge the PR stack	Six PRs chain off each other (#4, #5 → #6 → #8 → #10 → #12). main is still at the upstream commit.
subtract	The last v1 subcommand; 3 golden cases.

Running it

`./tests/run_golden.sh` diffs every subcommand against real bedtools
`python3 -m unittest discover -s tests` unit tests, no bedtools required
`ruff check .` lint, including the extensionless entry point